Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM302 Үйлдлийн систем

Бие даалт №1. The Sleeping Teaching Assistant manual

Шалгасан багш: Г.Ганбат /F.CS10/

Гүйцэтгэсэн оюутан: Б.Мөнхзаяа /B241940022/
Н.Сүх-Амар /B231940125/

Улаанбаатар хот
2026 он

Хэсэг 1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

Энэ програм нь **The Sleeping Teaching Assistant** бодлогыг (сурах бичгийн Synchronization Examples бүлгийн Programming Project) хэрэгжүүлдэг. Нэг TA, хязгаарлагдмал тооны хүлээх сандал, олон оюутны thread-ийг mutex ба POSIX semaphore-оор синхрончилсон C (Pthreads) програм нь thread-үүдийн **бодит event log** (CSV) гаргана. HTML UI тэр log-оор дараалсан ба TA-ийн төлвийг (унтаж байна / тусалж байна) алхам алхмаар дахин тоглуулан (animation) харуулна. Log ачаалмагц UI үүнийг автоматаар шалгаж (invariant: дараалсан урт ≤ сандал, FIFO, TA нэг дор нэг оюутанд тусална гэх мэт) ✔ эсвэл ✘ гэж мэдэгдэнэ.

C суулгаагүй үед туршихад зориулж UI-д ижил event форматтай **Demo** горим бий (C програмын загвар; бодит thread биш).

1.2 Дэмжих орчин

Зүйл	Шаардлага
Үйлдлийн систем	Linux (Ubuntu 22.04+, Debian 12+), macOS. Windows дээр C програмыг WSL (Ubuntu)-д ажиллуулна
Хөтөч (UI)	Chrome / Edge / Firefox / Safari-ийн шинэ хувилбар. Суулгах зүйлгүй, `src/sleeping_ta.html`-г нээхэд л болно
C орчин	gcc ≥ 9 эсвэл clang, GNU make, pthread. Туршсан: gcc 14.2.0, GNU Make 4.4.1 (Debian 13)
Node.js	Зөвхөн UI-ийн сонголтот тестэд (`make test-ui`). Туршсан: Node.js 20.19
Сүлжээ	Ажиллуулахад шаардлагагүй (зөвхөн `git clone`-д)

1.3 Суулгах ба нээх алхам

1. gcc болон make суусан эсэхийг шалгана: `gcc --version`, `make --version`. Debian/Ubuntu дээр суулгах: `sudo apt install build-essential`.
2. Кодыг татна: `git clone https://github.com/munkhzayabatochir997-droid/OS_BD1.git`, дараа нь `cd OS_BD1`.
3. Build: `make` (`build/sleeping_ta` болон `build/check_log` үүснэ).
4. Бодит log гаргана: `./build/sleeping_ta -c 3 -n 5 -p 1 -d 0.5 -r 3 -s 1 -o run.csv`.
5. `src/sleeping_ta.html`-г хөтөчид нээж, ”C-ийн log” талбараас `run.csv`-г сонгоод Play дарна.

Windows хэрэглэгч. C програм POSIX thread/semaphore ашигладаг тул Windows дээр шууд ажиллахгүй. Администратор эрхтэй PowerShell-д `wsl --install` ажиллуулж Ubuntu суулгаад, дээрх 1–4-р алхмыг **WSL-ийн дотор** (`~/` доор, `/mnt/c/...` биш) хийнэ. UI-г

Windows-ийн хөтөчид нээж, WSL-д үүсгэсэн `run.csv`-г сонгоно (Ubuntu-д `explorer.exe .` гэвэл тэр хавтас нээгдэнэ). C-гүйгээр шалгахдаа `examples/*.csv` (C-ийн бодит log) эсвэл UI-ийн Demo-г ашиглаж болно.

1.4 Оролт ба тохиргоо

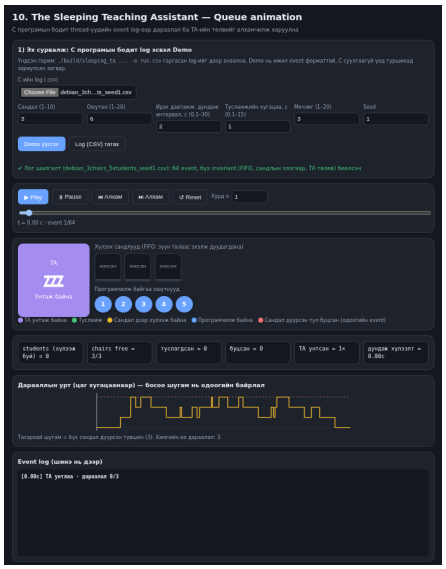
Оролт хоёр эх сурвалжтай: (1) **C-ийн бодит log** (үндсэн) ба (2) **Demo** (C-гүйгээр). UI-ийн дээд талын талбарууд (сандал, оюутан, ирэх давтамж, ...) нь зөвхөн **Demo**-гийн тохиргоо. C-ийн log ачаалахад эдгээр талбар өөрчлөгдөхгүй; log-ийн бодит тохиргоог сандлын тоо, оюутны дугуй, доорх тоолууранс харна.

C програмын аргумент (`./build/sleeping_ta ...`):

Аргумент	Утга	Муж
`-c`	Хүлээх сандлын тоо	1–10
`-n`	Оюутны тоо	1–20
`-p`	Ирэх давтамж: оюутан дахин ирэх дундаж интервал (с). Бага байх тусам олон ирнэ	0.01–60
`-d`	TA-ийн тусламжийн хугацаа (с)	0.01–60
`-r`	Оюутан бүрийн мөчлөгийн тоо	1–100
`-s`	Санамсаргүй тооны seed (ижил seed ⇒ ижил програмчлах хугацаа)	бүхэл тоо
`-o`	Event log (CSV) гаргах файл	файлын нэр
`-q`	Чимээгүй горим (stdout-д хэвлэхгүй)	-
`--no-mutex`	Mutex-гүй ажиллуулж race condition-ийг зориуд үзүүлнэ	-

Муж, төрлөөс гадуур утга өгвөл програм ойлгомжтой алдааны мэдээлэл хэвлээд **exit code 2**-оор гарна.

UI-ийн Demo тохиргоо: сандал 1–10 · оюутан 1–20 · ирэх давтамж 0.1–30 с · тусламжийн хугацаа 0.1–15 с · мөчлөг 1–20 · seed. Мужаас гадуур, хоосон, бүхэл биш утганд улаан алдааны мэдээлэл гарч, өмнөх дүрслэл өөрчлөгдөхгүй (Зураг 2).



Зураг 1. *Програмын нээсэн анхны байдал: run.csv ачаалсан, event 1, TA унтаж байна.*



Зураг 2. *Буруу оролт: Сандал = 0 үед улаан алдаа гарч, өмнөх дүрслэл хэвээр.*

1.5 Ажиллуулах, зогсоох, reset хийх

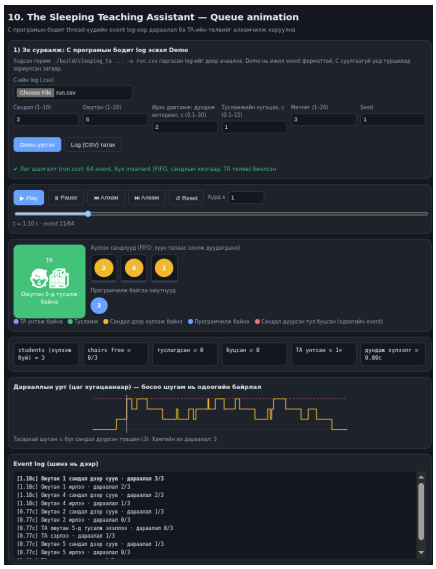
Удирдлага	Үйлдэл
Log (CSV) / файл сонгох	С-ийн log ачаална. Ачаалмагц автоматаар шалгаж ✔ / ✘ мэдэгдэнэ
Demo үүсгэх	Талбаруудын утгаар Demo event үүсгэнэ
Play / Pause	Event-үүдийг автоматаар тоглуулна / зогсооно
Алхам (өмнөх / дараах)	Нэг event-ээр ухарч / урагшилна. Нэг event бүрт нэг өөрчлөлт, event log-ийн дээд мөр шинэчлэгдэнэ
Slider	Дурын хугацаа руу шууд очно
Reset	Эхнээс (event 0) эхлүүлнэ
Хурд	0.25x–20x
Log татах	Demo-г CSV болгон татна

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв

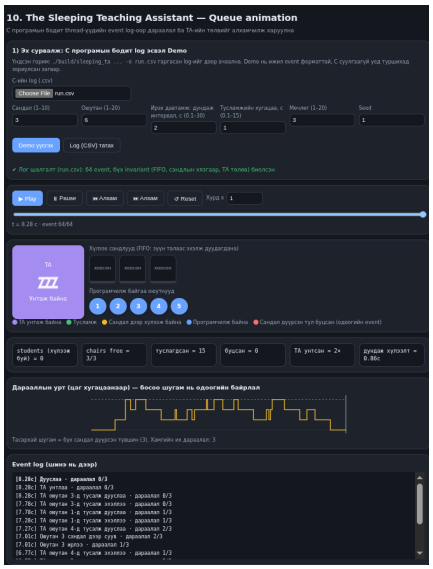
Тэмдэг	Утга
● Ягаан (TA)	TA унтаж байна (хүлээж буй оюутан байхгүй)
● Ногоон (TA)	TA оюутанд тусалж байна
● Цэнхэр (TA)	TA сэрүүн, оюутан дуудахад бэлэн
● Шар дугуй (сандал дээр)	Сандал дээр хүлээж буй оюутан. Зүүнээс баруун тийш FIFO буюу дуудагдах дараалал
● Цэнхэр дугуй (оюутан)	Програмчилж буй оюутан
● Улаан дугуй (оюутан)	Сандал дүүрсэн тул яг одоо буцсан оюутан
График (доор)	Дарааллын урт цаг хугацаанаар. Босоо шугам нь одоогийн event, тасархай шугам нь бүх сандал дүүрсэн түвшин
Тоолуур	Хүлээж буй (students semaphore-ийн утга), чөлөөт сандал, туслагдсан, буцсан, TA унтсан тоо, дундаж хүлээлт
Лог шалгалт	”✔ ... invariant биелсэн” эсвэл ”✘ Лог шалгалт: N зөрчил” (эхний 3 зөрчлийг харуулна)

1.7 Жишээнүүд

Жишээ 1. Дараалал үүсэх. `./build/sleeping_ta -c 3 -n 5 -p 1 -d 0.5 -r 3 -s 1 -o run.csv` (`examples/debian_3chairs_5students_seed1.csv`). TA ихэнхдээ завгүй, дараалал 2–3 хүрнэ. Зураг 3: event 11-д 3/3 сандал дүүрэн (дараалал 2, 4, 1), TA 5-р оюутанд тусалж байна. Зураг 4: event 64-т бүх event дууссан, 15 оюутан туслагдсан, буцсан 0, TA 2 удаа унтсан, ✔ invariant биелсэн.



Зураг 3. *Дараалал дүүрсэн үе (event 11, 3/3 сандал).*

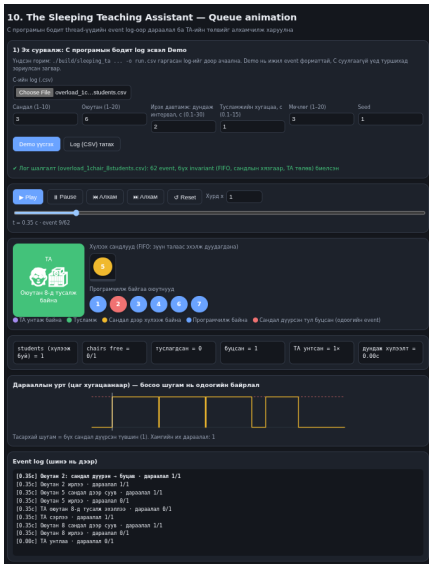


Зураг 4. *Дууссан төлөв (event 64): туслагдсан 15, буцсан 0, ✔ invariant.*

Жишээ 2. Сандал дүүрэх. `./build/sleeping_ta -c 1 -n 8 -p 0.5 -d 0.8 -r 3 -s 2 -o overload.csv` (`examples/overload_1chair_8students.csv`). Нэг сандал дүүрэн тул олон оюутан буцна (улаан дугуй, Зураг 5).

Жишээ 3. TA унтах. `./build/sleeping_ta -c 10 -n 3 -p 1 -d 0.4 -o idle.csv`. Дараалал бараг үүсэхгүй, TA байнга унтаж, сэрнэ.

Жишээ 4 (нэмэлт). Mutex-гүй үед race. `./build/sleeping_ta -c 1 -n 10 -p 0.05 -d 0.05 -r 3 -s 1 --no-mutex -o race.csv`. UI энэ log-ийг ачаалахад "✘ Лог шалгалт: N зөрчил" гарна (Зураг 6): нэг сандалтай үед дараалал 2 болсон. Энэ нь mutex зайлшгүй гэдгийг харуулна.



Зураг 5. *Overload log: 1 сандал дүүрэн, 2-р оюутан (улаан) буцсан.*



Зураг 6. *`--no-mutex` log: ✘ 43 зөрчил, дараалал сандлаас хэтэрсэн.*

1.8 Түгээмэл алдаа ба шийдэл

Шинж тэмдэг	Шалтгаан ба шийдэл
”Header дутуу”	Файл нь C програмын `-o` log биш. `-o run.csv`-ээр дахин үүсгэнэ
”Буруу мөр / event”	Log засагдсан эсвэл гэмтсэн. Дахин үүсгэнэ
”✘ Лог шалгалт”	Log дүрэм зөрчсөн (FIFO, сандлын хязгаар ...). Шалтгааныг эхний 3 зөрчлөөс харна
C: ”Алдаа: chairs (-c) нь 1-10 ...”	Аргументыг заасан мужид оруулна (exit code 2)
`make: command not found`	`sudo apt install build-essential`
`$'\r': command not found` (`run_tests.sh`)	Файл CRLF болсон. `sed -i 's/\r$//' tests/*.sh Makefile` эсвэл repo-г WSL дотор дахин clone хийнэ
`/mnt/c/...` дээр удаан эсвэл permission алдаа	Repo-г WSL-ийн `~/` доор clone хийнэ
Windows-д clone хийхэд `could not create work tree dir`	Controlled folder access / OneDrive хориглож байна. `C:\Users\<нэр>\projects` гэх мэт өөр хавтсанд clone хийнэ
PowerShell-д монгол үсэг эвдэрнэ	`Get-Content README.md -Encoding UTF8` эсвэл GitHub/VS Code-оор нээнэ

Хэсэг 2. Operational заавар

2.1 Repository clone хийх

```
git clone https://github.com/munkhzayabatochir997-droid/OS_BD1.git
cd OS_BD1
```

2.2 OS, compiler/runtime, version

Зүйл	Шаардлага	Баталгаажуулсан орчин
OS	Linux, macOS, Windows (WSL)	Debian 13 x86_64; CI: Ubuntu, macOS
C compiler	gcc ≥ 9 эсвэл clang (`-pthread`)	gcc 14.2.0
Build tool	GNU make	GNU Make 4.4.1
UI тест (сонголтоор)	Node.js	Node.js 20.19
Хөтөч	Chrome / Edge / Firefox / Safari	Chrome (UI screenshot)

Нээлттэй баталгаагүй зүйл: macOS-ийг гараар туршаагүй. Харин GitHub Actions дээр Ubuntu ба macOS (`macos-latest`)-ээр build ба автомат тест push бүрт ажилладаг. macOS

дээр `sem_init` дэмжигддэггүй тул код `#ifdef __APPLE__` дотор нэртэй семафор (`sem_open`) ашигладаг. Windows (MSVC) дээр pthread/semaphore байхгүй тул WSL ашиглана.

2.3 Dependency суулгах

Гадны сан шаардлагагүй. Зөвхөн стандарт C сан (libc), POSIX threads (`pthread`), POSIX semaphore (`semaphore.h`). UI нь цэвэр HTML/CSS/JavaScript, гадны сан эсвэл CDN ашигладаггүй. Node.js нь зөвхөн конголтот UI тестэд хэрэгтэй.

2.4 Build ба Run команд

Үйлдэл	Команд
Build	`make` (→ `build/sleeping_ta`, `build/check_log`)
Run	`./build/sleeping_ta -c 3 -n 5 -p 1 -d 0.5 -r 3 -s 1 -o run.csv`
Log шалгах	`./build/check_log run.csv` (invariant шалгагч, PASS / FAIL)
Автомат тест	`make test`
Stress тест	`make stress` (30 янзын тохиргоо, timeout нь deadlock илрүүлнэ)
Race шалгалт	`make tsan` (ThreadSanitizer)
UI model тест	`make test-ui` (Node.js)

Makefile ашиглахгүй бол гараар:

```
mkdir -p build
gcc -Wall -Wextra -O2 -pthread -o build/sleeping_ta src/sleeping_ta.c
gcc -Wall -Wextra -O2 -o build/check_log tests/check_log.c
./build/sleeping_ta -c 3 -n 5 -p 1 -d 0.5 -r 3 -s 1 -o run.csv
```

Хүлээгдэх үр дүн: build дуусахад `build/` дотор хоёр гүйцэтгэгдэх файл үүснэ, алдаа болон warning гарахгүй. Run нь event бүрийг хэвлэж, `run.csv` үүсгэнэ.

2.5 Үндсэн тестийн өгөгдөл ба хүлээгдсэн үр дүн

`make test` нь build хийж, `tests/run_tests.sh`-ийг ажиллуулна. Цаг хугацаатай (sleep) програм тул log яг ижил гарахгүй; иймд тест нь log-ийн **invariant**-ыг `check_log`-оор шалгана: дарааллын урт ≤ сандал · FIFO · TA нэг дор нэг оюутанд тусална · TA зөвхөн дараалал хоосон үед унтана · FULL зөвхөн сандал дүүрэн үед · оюутан бүр яг `rounds` удаа ирнэ · туслагдсан + буцсан = нийт ирэлт · END үед дараалал хоосон. Гаралтын төгсгөл: `Дүн: 22 амжилттай, 0 бүтэлгүйтсэн`.

ID	Оролт	Хүлээгдсэн	Жишээ гаралт
T1	`-c 3 -n 5 -p 0.3 -d 0.2 -r 3 -s 1`	invariant OK	PASS: helped=13, turned_away=2, max_queue=3
T2	`-c 1 -n 10 -p 0.1 -d 0.3 -r 2 -s 2`	FULL гарна	PASS: helped=4, turned_away=16
T3	`-c 10 -n 3 -p 0.3 -d 0.1 -r 3 -s 3`	FULL гарахгүй	PASS: helped=9, turned_away=0
T4	`-c 1 -n 1 -p 0.1 -d 0.1 -r 5 -s 4`	FULL гарахгүй	PASS: helped=5, ta_sleeps=6
T5	`-c 10 -n 20 -p 0.1 -d 0.05 -r 3 -s 5`	invariant OK	PASS: max_queue=10, 192 event
T6	`-c 2 -n 6 -p 0.2 -d 0.15 -r 4 -s 99`	invariant OK	PASS

2.6 Нэмэлт boundary / error тест

ID	Оролт	Хүлээгдсэн
E1–E12	Буруу оролт: `-c 0`, `-c 11`, `-c 3.5`, `-c abc`, `-n 0`, `-n 21`, `-p -1`, `-d 0`, `-r 0`, `-s -1`, үл мэдэгдэх сонголт, илүү аргумент	exit code 2 ба ойлгомжтой мэдээлэл
E13	Файл: `-o /nonexistent_dir/x.csv`	exit code 1
E14	`sleeping_ta --no-mutex --bogus`	exit code 2, алдааны мэдээлэл
ST1	`make stress`: 30 янзын c, n, seed (timeout 60 с)	бүгд invariant OK, timeout үгүй (30/30)
S1	`make tsan` (ThreadSanitizer)	data race мэдээлэлгүй, PASS
N1	`-c 1 -n 8 -p 0.05 -d 0.05 -r 3 -s K --no-mutex` (K = 1..8 хүртэл оролдоно)	`check_log` FAIL (сандлаас хэтэрсэн) эсвэл watchdog rc=3
N2	`--no-mutex`-ийн log-ийн эхний мөр	`mutex=0` бичигдсэн

Debian 13 дээр Manual-ийг өөрчлөлтгүй дагаж, цэвэр clone дээр хийсэн туршилт (бүрэн гаралт `docs/results/`-д):

Алхам	Команд / оролт	Үр дүн
Build	`make`	warning-гүй амжилттай
Run	`-c 3 -n 5 -p 1 -d 0.5 -r 3 -s 1`	ирэлт 15 = туслагдсан 15 + буцсан 0, дараалал ≤ 3, FIFO
Тест	`make test`	22 амжилттай, 0 бүтэлгүйтсэн
Stress, TSan	`make stress`, `make tsan`	30 ажиллуулалт 0 алдаа; data race илрээгүй
X1	Run-ийн log-ийг `check_log`-оор шалгах	PASS
X2	`-c 2 -n 20 -p 0.2 -d 0.5 -r 2 -s 7` (хэт ачаалал)	40 = 6 + 34 буцсан, дараалал ≤ 2, гацаагүй
X3	Ижил seed (42)-ээр 2 удаа	хураангуй үр дүн ижил
X4	`--no-mutex -c 1 -n 10`, seed 1–5	5/5 зөрчилтэй (29–259), 1 нь гацсан (watchdog)

Өөрийн өгөгдлөөр турших: `./build/sleeping_ta`-д өөр `-c -n -p -d -r -s` өгч, гарсан log-ийг `./build/check_log`-оор шалгаад UI-д ачаална.

2.7 Known issues

- Цаг хугацааны (sleep) програм тул ижил seed-тэй ч thread-үүдийн interleaving бага зэрэг өөр байж болно (програмчлах хугацаа ижил, event-ийн яг дараалал бага зэрэг зөрнө). Тиймээс тест нь log-ийн дүрэм (invariant)-ыг шалгадаг.
- `--no-mutex` горимд race магадлалтай тул заримдаа илрэхгүй. Race илрүүлэхийн тулд ”шалгах → суух” хоорондын цонхыг 1 мс-ээр зориуд өргөсгөдөг. Дараалал эвдэрч оюутан гацвал watchdog exit code 3-аар гаргана.
- UI-ийн Demo нь C програмын загвар (бодит thread биш); бодит нотолгоо нь C-ийн log.
- UI-д ”TA сэрүүн, хүлээж байна” ба ”програмчилж буй оюутан” хоёулаа цэнхэр өнгөтэй; legend-д TA-ийн сэрүүн төлөв тусдаа заагдаагүй.
- UI-ийн дүрслэлийг автоматаар шалгадаггүй (зөвхөн model, parser-ыг шалгана), UI screenshot-ыг гараар авсан.
- Windows (MSVC) дээр шууд ажиллахгүй, WSL шаардлагатай. macOS-ийг гараар туршаагүй (зөвхөн CI).